

PostgreSQL Beginner Interview Questions

100 beginner-level questions covering basic SQL, data types, constraints, simple queries, joins, DDL/DML basics.

Q1: What is PostgreSQL?

Difficulty: Beginner **Category:** General

Answer: PostgreSQL is an open-source, object-relational database management system (ORDBMS) that emphasizes extensibility and SQL standards compliance. It was originally developed at the University of California, Berkeley, and has been actively developed for over 30 years. PostgreSQL supports advanced data types, full ACID compliance, complex queries, foreign keys, triggers, views, and stored procedures. It is highly extensible, allowing users to define their own data types, operators, and functions. PostgreSQL runs on all major operating systems and is known for its robustness, reliability, and correctness.

Follow-up Questions:

- How does PostgreSQL differ from MySQL?
- What makes PostgreSQL "object-relational"?

What Interviewer Looks For: Understanding of what PostgreSQL is at a high level, its open-source nature, ACID compliance, and extensibility.

Common Mistakes: Confusing PostgreSQL with MySQL or saying it is purely relational without mentioning the object-relational features. Candidates who cannot explain ACID compliance demonstrate shallow knowledge.

Q2: What does ACID stand for in database systems?

Difficulty: Beginner **Category:** Transactions

Answer: ACID stands for Atomicity, Consistency, Isolation, and Durability. Atomicity means a transaction is treated as a single unit — either all operations succeed or none do. Consistency ensures a transaction brings the database from one valid state to another, maintaining all defined rules and constraints. Isolation means concurrent transactions execute as if they were serial, preventing interference between them. Durability guarantees that once a transaction is committed, it remains committed even in the event of a system failure. PostgreSQL is fully ACID compliant, which makes it suitable for mission-critical applications.

Follow-up Questions:

- What isolation levels does PostgreSQL support?
- How does PostgreSQL implement durability using WAL?

What Interviewer Looks For: Clear definitions of all four properties with practical examples of why each matters.

Common Mistakes: Memorizing the acronym without understanding what each property means in practice. Weak candidates cannot give a concrete example of an atomicity failure.

Q3: What is the difference between DDL and DML?

Difficulty: Beginner **Category:** SQL Basics

Answer: DDL (Data Definition Language) consists of SQL commands that define or modify the database structure. Common DDL commands include CREATE, ALTER, DROP, TRUNCATE, and RENAME. These commands affect the schema itself. DML (Data Manipulation Language) consists of commands that manipulate the data stored in the database. Common DML commands include SELECT, INSERT, UPDATE, DELETE, and MERGE. In PostgreSQL, DDL statements are transactional — they can be rolled back, which is a notable difference from some other databases like MySQL. DML statements are also transactional and follow ACID properties.

Follow-up Questions:

- What is DCL and TCL?
- Can you roll back a DROP TABLE in PostgreSQL?

What Interviewer Looks For: Clear distinction between schema-level and data-level operations, and awareness that PostgreSQL supports transactional DDL.

Common Mistakes: Saying DDL cannot be rolled back (true in MySQL/Oracle but not PostgreSQL). Forgetting that TRUNCATE is DDL, not DML.

Q4: What are the main data types available in PostgreSQL?

Difficulty: Beginner **Category:** Data Types

Answer: PostgreSQL offers a rich set of data types. Numeric types include INTEGER, BIGINT, SMALLINT, DECIMAL/NUMERIC, REAL, and DOUBLE PRECISION. Character types include CHAR(n), VARCHAR(n), and TEXT. Date/time types include DATE, TIME, TIMESTAMP, TIMESTAMPTZ, and INTERVAL. Boolean type is BOOLEAN. PostgreSQL also supports special types like UUID, JSON/JSONB, ARRAY, BYTEA (binary data), INET/CIDR (network addresses), and geometric types (POINT, LINE, POLYGON). The SERIAL and BIGSERIAL pseudo-types are used for auto-incrementing integers. Additionally, PostgreSQL allows users to create custom types.

Follow-up Questions:

- What is the difference between VARCHAR and TEXT in PostgreSQL?
- When would you use JSONB over JSON?

What Interviewer Looks For: Familiarity with core types and awareness of PostgreSQL-specific types like JSONB, ARRAY, UUID.

Common Mistakes: Listing only basic types without mentioning PostgreSQL-specific ones. Not knowing that TEXT and VARCHAR have no performance difference in PostgreSQL.

Q5: What is the difference between CHAR, VARCHAR, and TEXT in PostgreSQL?

Difficulty: Beginner **Category:** Data Types

Answer: CHAR(n) is a fixed-length character type that pads shorter strings with spaces to reach the specified length. VARCHAR(n) is a variable-length character type with an optional maximum length limit. TEXT is a variable-length character type with no length limit. In PostgreSQL, there is no performance difference between VARCHAR and TEXT — they are stored identically. CHAR(n) can actually be slightly less efficient due to the padding behavior. The main practical difference is that VARCHAR(n) enforces a maximum length constraint, which can be useful for validation. For most use cases in PostgreSQL, TEXT is preferred because of its flexibility and equivalent performance.

Follow-up Questions:

- Is there a performance difference between VARCHAR and TEXT in PostgreSQL?
- What happens when you insert a string longer than CHAR(n)?

What Interviewer Looks For: Understanding that TEXT and VARCHAR are equivalent in PostgreSQL performance-wise, and knowing when CHAR is actually appropriate.

Common Mistakes: Saying VARCHAR is faster than TEXT, or that CHAR is faster than VARCHAR. These are myths from other database systems that do not apply to PostgreSQL.

Q6: What is a PRIMARY KEY?

Difficulty: Beginner **Category:** Constraints

Answer: A PRIMARY KEY is a column or combination of columns that uniquely identifies each row in a table. It combines the NOT NULL and UNIQUE constraints — every primary key value must be non-null and must be unique across all rows. A table can have only one primary key. In PostgreSQL, a PRIMARY KEY automatically creates a unique B-tree index on the key column(s). Primary keys are fundamental to relational database design as they provide a reliable way to reference rows in other tables through foreign keys. When using composite primary keys, the combination of all key columns must be unique, though individual columns may repeat.

Follow-up Questions:

- What is the difference between a PRIMARY KEY and a UNIQUE constraint?
- Can a PRIMARY KEY consist of multiple columns?

What Interviewer Looks For: Understanding of uniqueness and NOT NULL requirements, the automatic index creation, and the role of PKs in referential integrity.

Common Mistakes: Not knowing that PRIMARY KEY implies NOT NULL. Confusing the difference between a single-column and composite primary key.

Q7: What is a FOREIGN KEY?

Difficulty: Beginner **Category:** Constraints

Answer: A FOREIGN KEY is a column or set of columns in one table that references the PRIMARY KEY (or UNIQUE key) of another table, establishing a relationship between the two tables. The table with the foreign key is called the child table, and the referenced table is the parent table. Foreign keys enforce referential integrity — they ensure that a value in the child table always corresponds to a valid value in the parent table. PostgreSQL supports ON DELETE and ON UPDATE actions: CASCADE, SET NULL, SET DEFAULT, and RESTRICT/NO ACTION. Attempting to insert a value in the foreign key column that doesn't exist in the referenced table will result in an error.

Follow-up Questions:

- What is the difference between ON DELETE CASCADE and ON DELETE SET NULL?
- What happens when you try to delete a parent row that has child rows referencing it?

What Interviewer Looks For: Understanding of referential integrity, parent-child relationships, and the cascade options.

Common Mistakes: Not understanding the directionality (child references parent). Not knowing the difference between CASCADE, SET NULL, and RESTRICT.

Q8: What is the difference between UNIQUE and PRIMARY KEY constraints?

Difficulty: Beginner **Category:** Constraints

Answer: Both UNIQUE and PRIMARY KEY constraints ensure that all values in a column (or set of columns) are distinct. However, there are key differences. A table can have only one PRIMARY KEY but multiple UNIQUE constraints. A PRIMARY KEY column cannot contain NULL values, whereas a UNIQUE constraint allows NULLs (and in PostgreSQL, multiple NULLs are allowed in a UNIQUE column because NULL is not equal to NULL). A PRIMARY KEY is intended to be the canonical identifier for rows, while UNIQUE constraints enforce business rules like ensuring no duplicate email addresses. Both create an index automatically in PostgreSQL.

Follow-up Questions:

- Can a UNIQUE column have NULL values in PostgreSQL?
- Can you use a UNIQUE column as a foreign key target?

What Interviewer Looks For: Clarity on NULL handling differences and the conceptual purpose difference between the two constraints.

Common Mistakes: Saying UNIQUE columns cannot have NULLs. Not knowing that a PRIMARY KEY is essentially UNIQUE + NOT NULL.

Q9: What is a NOT NULL constraint?

Difficulty: Beginner **Category:** Constraints

Answer: A NOT NULL constraint ensures that a column cannot store a NULL value. When a column is defined with NOT NULL, any INSERT or UPDATE that tries to put a NULL into that column will fail with an error. This constraint is important for data integrity because NULL represents an unknown or missing value, and some columns should always have a known value (like a user's email, a price, or a creation timestamp). In PostgreSQL, you can add a NOT NULL constraint to an existing column using ALTER TABLE, but this will fail if any existing rows have NULL in that column. The PRIMARY KEY constraint implicitly includes NOT NULL.

Follow-up Questions:

- What is the difference between NULL and an empty string?
- How do you handle NULL values in aggregate functions?

What Interviewer Looks For: Understanding that NULL means "unknown" not "zero" or "empty", and when NOT NULL is appropriate.

Common Mistakes: Confusing NULL with zero or empty string. Not knowing that aggregate functions like COUNT, SUM ignore NULLs.

Q10: What is a CHECK constraint?

Difficulty: Beginner **Category:** Constraints

Answer: A CHECK constraint specifies a boolean expression that must evaluate to TRUE for any row being inserted or updated. It allows you to enforce domain integrity — ensuring that data meets specific business rules at the database level. For example, you could add CHECK (age >= 0) to ensure age is never negative, or CHECK (price > 0) for a products table. CHECK constraints can reference multiple columns in the same row. In

PostgreSQL, CHECK constraints are not validated for existing rows when you add them to an existing table unless you use the NOT VALID option followed by VALIDATE CONSTRAINT. A CHECK constraint that evaluates to NULL is considered to pass.

Follow-up Questions:

- Can a CHECK constraint reference another table?
- What happens when a CHECK constraint evaluates to NULL?

What Interviewer Looks For: Understanding of when to use CHECK vs. application-level validation, and the NULL behavior.

Common Mistakes: Trying to use CHECK to reference another table (use a foreign key instead). Not knowing that NULL in a CHECK constraint passes the constraint.

Q11: What is the SELECT statement and what is its basic syntax?

Difficulty: Beginner **Category:** Basic SQL

Answer: The SELECT statement is used to retrieve data from one or more tables in a database. Its basic syntax is: `SELECT column1, column2 FROM table_name WHERE condition ORDER BY column ASC/DESC LIMIT n`. The FROM clause specifies which table(s) to read from. The WHERE clause filters rows based on conditions. ORDER BY sorts the results. LIMIT restricts the number of rows returned. You can use `SELECT *` to retrieve all columns, though this is generally discouraged in production code for performance reasons. The SELECT statement is the foundation of SQL querying and supports a wide variety of clauses including GROUP BY, HAVING, DISTINCT, and JOIN.

Follow-up Questions:

- What is the order of clause execution in a SELECT statement?
- What is the difference between WHERE and HAVING?

What Interviewer Looks For: Knowledge of clause syntax, logical order of operations, and ability to write correct queries.

Common Mistakes: Not knowing the logical execution order (FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT). Writing WHERE after GROUP BY when HAVING should be used.

Q12: What is the logical order of SQL query execution?

Difficulty: Beginner **Category:** Basic SQL

Answer: The logical order of SQL query execution differs from the written order. The execution order is: 1) FROM and JOINS — tables are identified and joined; 2) WHERE — rows are filtered; 3) GROUP BY — rows are grouped; 4) HAVING — groups are filtered; 5) SELECT — columns are selected and expressions evaluated; 6) DISTINCT — duplicates removed; 7) ORDER BY — results sorted; 8) LIMIT/OFFSET — rows restricted. This is important because column aliases defined in SELECT are not available in WHERE (since WHERE executes before SELECT). Understanding this order helps diagnose query errors and write correct queries.

Follow-up Questions:

- Why can't you use a SELECT alias in a WHERE clause?
- Can you use a SELECT alias in ORDER BY?

What Interviewer Looks For: Clear understanding of the logical execution order and its practical implications.

Common Mistakes: Thinking SQL executes in the written order. Trying to use a SELECT alias in WHERE and not understanding why it fails (though PostgreSQL sometimes allows it in ORDER BY as a special case).

Q13: What is the difference between WHERE and HAVING?

Difficulty: Beginner **Category:** Basic SQL

Answer: WHERE filters individual rows before any grouping occurs, while HAVING filters groups after GROUP BY has been applied. WHERE cannot reference aggregate functions (like COUNT, SUM, AVG) because aggregation hasn't happened yet at that stage. HAVING can reference aggregate functions. For example, to find departments with more than 5 employees: `SELECT department, COUNT() FROM employees GROUP BY department HAVING COUNT() > 5`. You cannot put `COUNT(*) > 5` in a WHERE clause. WHERE operates on raw data, HAVING operates on aggregated results. For performance, it's better to filter as much as possible in WHERE before grouping.

Follow-up Questions:

- Can you use HAVING without GROUP BY?
- Which is evaluated first, WHERE or HAVING?

What Interviewer Looks For: Ability to articulate the execution order difference and give a concrete example.

Common Mistakes: Using HAVING when WHERE would work (less efficient). Trying to use aggregate functions in WHERE.

Q14: What are JOINS in SQL? Explain the different types.

Difficulty: Beginner **Category:** Joins

Answer: JOINS combine rows from two or more tables based on a related column. The main types are: INNER JOIN returns only rows where there is a match in both tables. LEFT (OUTER) JOIN returns all rows from the left table and matching rows from the right table; unmatched right rows return NULL. RIGHT (OUTER) JOIN is the mirror of LEFT JOIN — all rows from the right table. FULL (OUTER) JOIN returns all rows from both tables, with NULLs where there is no match. CROSS JOIN returns the Cartesian product — every combination of rows from both tables. SELF JOIN joins a table to itself, useful for hierarchical data. LEFT and RIGHT JOINS are the most common after INNER JOIN.

Follow-up Questions:

- What is a CROSS JOIN and when would you use it?
- How do you write a SELF JOIN?

What Interviewer Looks For: Clear explanation of each join type with understanding of when NULLs appear in the result.

Common Mistakes: Confusing LEFT JOIN result with INNER JOIN. Not knowing FULL OUTER JOIN exists. Forgetting that unmatched rows in outer joins produce NULLs.

Q15: What is an INNER JOIN?

Difficulty: Beginner **Category:** Joins

Answer: An INNER JOIN returns only the rows that have matching values in both tables based on the join condition. If a row in the left table has no corresponding match in the right table, that row is excluded from the result, and vice versa. The syntax is: `SELECT a.col, b.col FROM table_a a INNER JOIN table_b b ON a.id = b.a_id`. INNER JOIN is the default join type — writing JOIN without a qualifier is equivalent to INNER JOIN. For example, joining orders and customers where only orders that have a matching customer are returned. It is the most commonly used join type in practice.

Follow-up Questions:

- Can INNER JOIN return duplicate rows?
- How is INNER JOIN different from a WHERE clause with multiple tables?

What Interviewer Looks For: Understanding of the filtering behavior and ability to write the syntax correctly.

Common Mistakes: Not realizing that INNER JOIN can return duplicate rows if the join condition matches multiple rows. Thinking JOIN always means INNER JOIN in all databases.

Q16: What is a LEFT JOIN?

Difficulty: Beginner **Category:** Joins

Answer: A LEFT JOIN (or LEFT OUTER JOIN) returns all rows from the left table, plus the matching rows from the right table. For rows in the left table that have no match in the right table, NULL values are returned for all right table columns. This is useful when you want to see all records from one table regardless of whether they have related records in another table — for example, finding all customers including those who have never placed an order: `SELECT c.name, o.order_id FROM customers c LEFT JOIN orders o ON c.id = o.customer_id`. Customers with no orders will appear with NULL in the order_id column.

Follow-up Questions:

- How do you find rows in the left table that have NO match in the right table?
- What is the difference between LEFT JOIN and LEFT OUTER JOIN?

What Interviewer Looks For: Ability to use LEFT JOIN to find unmatched rows by adding `WHERE right_table.id IS NULL`.

Common Mistakes: Not knowing that LEFT JOIN and LEFT OUTER JOIN are identical. Not knowing the anti-join pattern (`WHERE right.id IS NULL`).

Q17: What is the difference between UNION and UNION ALL?

Difficulty: Beginner **Category:** Set Operations

Answer: Both UNION and UNION ALL combine the result sets of two or more SELECT queries into a single result. The key difference is that UNION removes duplicate rows from the combined result, while UNION ALL keeps all rows including duplicates. UNION ALL is faster because it does not need to sort and deduplicate the results. If you know the result sets are mutually exclusive (no duplicates possible), always use UNION ALL for better performance. Both require the same number of columns and compatible data types in corresponding columns. UNION is equivalent to UNION ALL followed by a DISTINCT operation.

Follow-up Questions:

- What other set operations does PostgreSQL support besides UNION?
- When would you use INTERSECT?

What Interviewer Looks For: Clear understanding of the deduplication difference and the performance implication of choosing UNION ALL.

Common Mistakes: Using UNION when UNION ALL is sufficient and faster. Not knowing about INTERSECT and EXCEPT.

Q18: What is the GROUP BY clause?

Difficulty: Beginner **Category:** Basic SQL

Answer: GROUP BY groups rows that have the same values in specified columns into summary rows. It is typically used with aggregate functions like COUNT, SUM, AVG, MIN, and MAX to produce one result row per group. For example: SELECT department, COUNT(*) as emp_count, AVG(salary) as avg_salary FROM employees GROUP BY department. This returns one row per department with the count and average salary. Every column in the SELECT list must either appear in the GROUP BY clause or be used within an aggregate function. PostgreSQL also supports GROUPING SETS, ROLLUP, and CUBE for more complex grouping scenarios.

Follow-up Questions:

- What is GROUPING SETS?
- Can you GROUP BY a column not in SELECT?

What Interviewer Looks For: Understanding of the rule that non-aggregated columns must appear in GROUP BY, and practical use of aggregate functions.

Common Mistakes: Trying to SELECT a non-aggregated column without including it in GROUP BY. Not knowing about ROLLUP or CUBE.

Q19: What are aggregate functions in PostgreSQL?

Difficulty: Beginner **Category:** Basic SQL

Answer: Aggregate functions perform a calculation on a set of rows and return a single value. The main aggregate functions in PostgreSQL are: COUNT() — counts rows (COUNT() counts all rows, COUNT(col) counts non-NULL values); SUM() — adds up all values; AVG() — calculates the average; MIN() — returns the minimum value; MAX() — returns the maximum value. PostgreSQL also provides statistical aggregates like STDDEV(), VARIANCE(), and STRING_AGG() for concatenating strings. Aggregate functions ignore NULL values except COUNT(). They are typically used with GROUP BY but can also be used without it to aggregate the entire table.

Follow-up Questions:

- What is the difference between COUNT(*) and COUNT(column_name)?
- How does STRING_AGG work?

What Interviewer Looks For: Knowing the core aggregate functions, NULL handling, and that COUNT(*) vs COUNT(col) behaves differently.

Common Mistakes: Not knowing that COUNT(column) skips NULLs while COUNT(*) counts all rows. Forgetting about string aggregation functions.

Q20: What is the ORDER BY clause?

Difficulty: Beginner **Category:** Basic SQL

Answer: ORDER BY sorts the result set of a query by one or more columns. By default, it sorts in ascending order (ASC). You can specify DESC for descending order. You can sort by multiple columns: ORDER BY col1 ASC, col2 DESC — rows are first sorted by col1, then by col2 for ties. In PostgreSQL, NULL values are treated as larger than non-NULL values by default (appearing last in ASC order). You can control NULL placement with NULLS FIRST or NULLS LAST. ORDER BY can reference column aliases defined in SELECT, which is an exception to the general rule that SELECT aliases are not available in earlier clauses.

Follow-up Questions:

- How does PostgreSQL order NULL values by default?
- Can you ORDER BY a column that is not in SELECT?

What Interviewer Looks For: Multi-column sorting, NULL handling, and the NULLS FIRST/LAST option.

Common Mistakes: Not knowing about NULLS FIRST/LAST. Forgetting that ORDER BY is the last logical step in query execution.

Q21: What is the LIMIT and OFFSET clause?

Difficulty: Beginner **Category:** Basic SQL

Answer: LIMIT restricts the number of rows returned by a query. OFFSET skips a specified number of rows before beginning to return rows. Together they are used for pagination: LIMIT 10 OFFSET 20 returns rows 21-30. For example: SELECT * FROM products ORDER BY price DESC LIMIT 5 returns the 5 most expensive products. It is important to always use ORDER BY with LIMIT to get deterministic results — without ORDER BY, the database may return any rows. The performance of large OFFSET values can be poor because PostgreSQL must still scan and discard the offset rows.

Follow-up Questions:

- Why is large OFFSET inefficient?
- What is a better approach to pagination than LIMIT/OFFSET?

What Interviewer Looks For: Understanding of pagination pattern and the performance problem with large offsets (keyset/cursor-based pagination is the answer).

Common Mistakes: Using LIMIT without ORDER BY. Not knowing about the performance degradation with large offsets and cursor-based alternatives.

Q22: What is the DISTINCT keyword?

Difficulty: Beginner **Category:** Basic SQL

Answer: DISTINCT eliminates duplicate rows from the result set. SELECT DISTINCT returns only unique rows. For example: SELECT DISTINCT country FROM customers returns each country only once. DISTINCT applies to the entire row — it considers all selected columns when determining uniqueness. DISTINCT ON is a PostgreSQL-specific extension that returns one row per distinct group of specified columns (like a simplified GROUP BY with LIMIT 1 per group). DISTINCT has a performance cost because it requires sorting or hashing to identify and remove duplicates, similar to UNION.

Follow-up Questions:

- What is DISTINCT ON in PostgreSQL?
- How does DISTINCT perform compared to GROUP BY for deduplication?

What Interviewer Looks For: Understanding that DISTINCT applies to the whole row and knowing about the PostgreSQL-specific DISTINCT ON.

Common Mistakes: Thinking DISTINCT(col) is a function call — it is not. Not knowing that DISTINCT ON is a PostgreSQL extension not available in standard SQL.

Q23: How do you create a table in PostgreSQL?

Difficulty: Beginner **Category:** DDL

Answer: You create a table using the CREATE TABLE statement: CREATE TABLE table_name (column1 datatype constraints, column2 datatype constraints, ...); For example:

```
CREATE TABLE employees (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    email VARCHAR(255) UNIQUE NOT NULL,  
    salary DECIMAL(10,2) CHECK (salary > 0),  
    department_id INTEGER REFERENCES departments(id),  
    created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

You can specify constraints inline (column-level) or as separate table-level constraints. CREATE TABLE IF NOT EXISTS prevents an error if the table already exists. You can also use CREATE TABLE AS SELECT to create a table from query results.

Follow-up Questions:

- What is CREATE TABLE AS SELECT?
- How do you create a temporary table?

What Interviewer Looks For: Ability to write a complete CREATE TABLE with appropriate data types and constraints.

Common Mistakes: Forgetting to add constraints. Not knowing about SERIAL for auto-increment. Using the wrong data type for monetary values (use NUMERIC, not FLOAT).

Q24: How do you INSERT data into a table?

Difficulty: Beginner **Category:** DML

Answer: Data is inserted using the INSERT INTO statement. Single row: INSERT INTO table_name (col1, col2) VALUES (val1, val2). Multiple rows: INSERT INTO table_name (col1, col2) VALUES (val1, val2), (val3, val4). You can omit column names if providing values for all columns in order, but it is best practice to always specify column names. PostgreSQL also supports INSERT ... SELECT to insert data from a query. The RETURNING clause is a PostgreSQL feature that returns values from the inserted row — very useful for getting auto-generated IDs: INSERT INTO orders (customer_id) VALUES (1) RETURNING id.

Follow-up Questions:

- What is INSERT ON CONFLICT (upsert)?
- How does the RETURNING clause work?

What Interviewer Looks For: Multi-row insert syntax, the RETURNING clause, and awareness of INSERT ON CONFLICT for upserts.

Common Mistakes: Not specifying column names. Not knowing about the RETURNING clause for getting generated values.

Q25: How do you UPDATE records in PostgreSQL?

Difficulty: Beginner **Category:** DML

Answer: The UPDATE statement modifies existing rows: UPDATE table_name SET col1 = val1, col2 = val2 WHERE condition. The WHERE clause is critical — without it, all rows in the table are updated. For example: UPDATE employees SET salary = salary * 1.1 WHERE department = 'Engineering'. PostgreSQL supports UPDATE with a FROM clause for joining to another table: UPDATE employees SET salary = s.new_salary FROM salary_adjustments s WHERE employees.id = s.employee_id. The RETURNING clause also works with UPDATE to return the modified rows. Always double-check your WHERE clause before running UPDATE in production.

Follow-up Questions:

- How do you UPDATE using data from another table?
- What does RETURNING do in an UPDATE statement?

What Interviewer Looks For: Importance of the WHERE clause, UPDATE FROM syntax, and RETURNING.

Common Mistakes: Forgetting the WHERE clause and updating all rows. Not knowing about UPDATE...FROM for joining another table.

Q26: How do you DELETE records in PostgreSQL?

Difficulty: Beginner **Category:** DML

Answer: DELETE removes rows from a table: DELETE FROM table_name WHERE condition. Without a WHERE clause, all rows are deleted (but the table structure remains — unlike DROP TABLE). For example: DELETE FROM orders WHERE created_at < NOW() - INTERVAL '1 year'. The RETURNING clause works with DELETE to return deleted rows. DELETE is slower than TRUNCATE for removing all rows because it generates individual row-level WAL entries and fires triggers. To delete rows based on a join, use DELETE...USING: DELETE FROM employees USING departments d WHERE employees.dept_id = d.id AND d.name = 'Old Dept'.

Follow-up Questions:

- What is the difference between DELETE, TRUNCATE, and DROP?
- How do you delete rows based on a condition from another table?

What Interviewer Looks For: Knowing that DELETE without WHERE removes all rows, difference from TRUNCATE, and RETURNING.

Common Mistakes: Confusing DELETE (removes rows) with DROP (removes table). Not knowing DELETE...USING syntax for joining another table.

Q27: What is the difference between DELETE and TRUNCATE?

Difficulty: Beginner **Category:** DDL/DML

Answer: DELETE removes rows one by one and logs each deletion in WAL (Write-Ahead Log), making it slower but allowing WHERE filtering and triggers to fire. TRUNCATE removes all rows at once, is much faster, resets sequences (when using RESTART IDENTITY), and does not fire row-level triggers. TRUNCATE is DDL in PostgreSQL and is transactional (can be rolled back), unlike in some other databases. DELETE can have a WHERE clause to remove specific rows; TRUNCATE cannot. For resetting a table to empty, TRUNCATE is preferred for performance. TRUNCATE can also cascade to child tables with TRUNCATE ... CASCADE.

Follow-up Questions:

- Can TRUNCATE be rolled back in PostgreSQL?
- Does TRUNCATE reset sequences automatically?

What Interviewer Looks For: Performance difference, WAL behavior, trigger firing, and the TRUNCATE RESTART IDENTITY option.

Common Mistakes: Saying TRUNCATE cannot be rolled back (it can in PostgreSQL). Not knowing TRUNCATE resets sequences with RESTART IDENTITY.

Q28: What is a schema in PostgreSQL?

Difficulty: Beginner **Category:** Database Objects

Answer: A schema in PostgreSQL is a namespace that contains database objects such as tables, views, indexes, sequences, and functions. It is different from a "schema" meaning table structure. Every database has a default schema called `public`. Schemas allow you to organize database objects into logical groups, prevent name conflicts between objects owned by different users, and control access permissions. The `search_path` variable determines which schemas PostgreSQL looks in when you reference an object without a schema prefix. You create schemas with `CREATE SCHEMA schema_name`. You reference objects with `schema.object` notation, e.g., `hr.employees`.

Follow-up Questions:

- What is `search_path` and how does it work?
- How are schemas different from databases in PostgreSQL?

What Interviewer Looks For: Understanding of schemas as namespaces, the public schema, `search_path`, and the difference from databases.

Common Mistakes: Confusing schema (namespace) with table schema (structure). Not knowing about `search_path`.

Q29: What is a sequence in PostgreSQL?

Difficulty: Beginner **Category:** Database Objects

Answer: A sequence is a database object that generates a series of unique numeric values, typically used for auto-incrementing primary keys. You create sequences with `CREATE SEQUENCE`, but usually they are created implicitly when you use the `SERIAL` or `BIGSERIAL` data types. Key sequence functions: `nextval('seq_name')` gets the next value, `currval('seq_name')` returns the last value fetched in the current session, `setval('seq_name', n)` sets the sequence to a specific value. Sequences are not rolled back with transactions — if a transaction fails, the sequence number is still incremented, causing gaps. This is intentional for performance.

Follow-up Questions:

- Why are there gaps in SERIAL columns?
- What is the difference between SERIAL and IDENTITY columns?

What Interviewer Looks For: Understanding that sequences are non-transactional (gaps on rollback), and that SERIAL is syntactic sugar for sequence + default.

Common Mistakes: Expecting no gaps in auto-increment sequences. Not knowing about the newer GENERATED ALWAYS AS IDENTITY syntax introduced in PostgreSQL 10.

Q30: What is the difference between SERIAL and IDENTITY columns?

Difficulty: Beginner **Category:** Data Types

Answer: SERIAL is a PostgreSQL-specific pseudo-type that creates a sequence and sets it as the column's default. It is not part of SQL standard. GENERATED AS IDENTITY (introduced in PostgreSQL 10, part of SQL:2003 standard) is the recommended modern approach. GENERATED ALWAYS AS IDENTITY prevents explicit inserts of a value unless you use OVERRIDING SYSTEM VALUE. GENERATED BY DEFAULT AS IDENTITY allows explicit values. Identity columns are cleaner because the sequence is directly associated with the column, while SERIAL just sets a default and the sequence can be independently modified. Both create auto-incrementing integer columns in practice.

Follow-up Questions:

- Can you insert an explicit value into a GENERATED ALWAYS AS IDENTITY column?
- How do you reset an identity column sequence?

What Interviewer Looks For: Awareness of the SQL standard approach and when GENERATED ALWAYS vs BY DEFAULT matters.

Common Mistakes: Still using SERIAL in new code without knowing IDENTITY exists. Not knowing the difference between ALWAYS and BY DEFAULT.

Q31: What is an INDEX in PostgreSQL?

Difficulty: Beginner **Category:** Indexes

Answer: An index is a data structure that improves the speed of data retrieval operations at the cost of additional storage and slower write operations. Without an index, PostgreSQL performs a sequential scan — reading every row in the table. With an index on a frequently queried column, PostgreSQL can locate rows much faster. The default index type is B-tree, which works for equality, range, and sorting queries. You create an index with: CREATE INDEX idx_name ON table_name (column_name). PostgreSQL automatically creates indexes for PRIMARY KEY and UNIQUE constraints. Indexes are maintained automatically on INSERT, UPDATE, and DELETE.

Follow-up Questions:

- What are the different index types in PostgreSQL?
- When would an index actually hurt performance?

What Interviewer Looks For: Understanding of sequential scan vs index scan tradeoff, and that indexes have write overhead.

Common Mistakes: Thinking more indexes always help. Not knowing that small tables or low-cardinality columns often don't benefit from indexes.

Q32: What is a VIEW in PostgreSQL?

Difficulty: Beginner **Category:** Database Objects

Answer: A VIEW is a virtual table defined by a stored SELECT query. It does not store data itself — each time you query a view, the underlying query is executed. Views simplify complex queries by giving them a name, enhance security by exposing only certain columns, and provide an abstraction layer so underlying table structure can change without breaking client queries. Syntax: `CREATE VIEW view_name AS SELECT ...;` You can query a view like a table: `SELECT * FROM view_name`. PostgreSQL also supports MATERIALIZED VIEWS, which do store the result and must be refreshed manually. Simple views can be updatable (INSERT/UPDATE/DELETE pass through to the base table).

Follow-up Questions:

- What is a materialized view?
- When can you INSERT into a view?

What Interviewer Looks For: Understanding that views are virtual, knowing materialized views exist, and when views are updatable.

Common Mistakes: Thinking views store data (regular views do not). Not knowing about materialized views.

Q33: What is a transaction in PostgreSQL?

Difficulty: Beginner **Category:** Transactions

Answer: A transaction is a sequence of SQL statements that are executed as a single unit of work. Transactions begin with `BEGIN` (or `START TRANSACTION`), end with `COMMIT` (to save changes) or `ROLLBACK` (to undo changes). All statements within a transaction see a consistent snapshot of the data. If any statement fails, you can roll back the entire transaction. PostgreSQL runs every statement in a transaction automatically (autocommit mode) — each individual statement is its own transaction unless you explicitly use `BEGIN`. Savepoints allow partial rollbacks within a transaction: `SAVEPOINT sp1; ... ROLLBACK TO sp1`.

Follow-up Questions:

- What are savepoints?
- What happens to a transaction when a statement errors inside it?

What Interviewer Looks For: Understanding of `BEGIN/COMMIT/ROLLBACK`, autocommit behavior, and savepoints.

Common Mistakes: Not knowing PostgreSQL runs in autocommit mode by default. Not knowing about savepoints for partial rollbacks.

Q34: What is NULL in SQL and how do you check for it?

Difficulty: Beginner **Category:** Basic SQL

Answer: NULL represents the absence of a value or an unknown value. It is not the same as zero, empty string, or false. NULL is not equal to anything, including itself — `NULL = NULL` evaluates to NULL (unknown), not TRUE. To check for NULL, you must use `IS NULL` or `IS NOT NULL` operators: `SELECT * FROM employees WHERE manager_id IS NULL`. Arithmetic operations with NULL return NULL. Most aggregate functions ignore NULLs (except `COUNT(*)`). The `COALESCE` function returns the first non-NULL argument: `COALESCE(value, 'default')`. The `NULLIF` function returns NULL if two values are equal.

Follow-up Questions:

- What does NULL = NULL evaluate to in SQL?
- What is COALESCE and how is it different from NULLIF?

What Interviewer Looks For: Understanding that NULL is "unknown" and requires IS NULL, not = NULL.

Common Mistakes: Writing WHERE column = NULL instead of WHERE column IS NULL. Not knowing that NULL propagates through most operations.

Q35: What is the COALESCE function?

Difficulty: Beginner **Category:** Functions

Answer: COALESCE returns the first non-NULL value in a list of arguments. It takes any number of arguments: COALESCE(val1, val2, val3, ...). It is commonly used to provide default values for NULL columns: SELECT name, COALESCE(phone, 'No phone') FROM contacts. It can also be used for conditional logic: COALESCE(discount_price, regular_price). COALESCE is equivalent to a CASE WHEN val1 IS NOT NULL THEN val1 WHEN val2 IS NOT NULL THEN val2 ... ELSE NULL END. It short-circuits — once a non-NULL value is found, subsequent arguments are not evaluated. It is SQL-standard and works across all databases.

Follow-up Questions:

- How is COALESCE different from NULLIF?
- What is the NVL function (Oracle equivalent)?

What Interviewer Looks For: Practical use cases for COALESCE in handling NULLs, and the short-circuit behavior.

Common Mistakes: Not knowing COALESCE can take more than two arguments. Confusing with NULLIF which returns NULL when two values are equal.

Q36: What is the CASE expression in SQL?

Difficulty: Beginner **Category:** Basic SQL

Answer: CASE is a conditional expression that works like an if-then-else statement. There are two forms. Simple CASE: CASE column WHEN val1 THEN result1 WHEN val2 THEN result2 ELSE default END. Searched CASE: CASE WHEN condition1 THEN result1 WHEN condition2 THEN result2 ELSE default END. The ELSE clause is optional; if omitted and no condition matches, NULL is returned. CASE can be used in SELECT, WHERE, ORDER BY, GROUP BY, and aggregate functions. Example: SELECT name, CASE WHEN salary > 100000 THEN 'High' WHEN salary > 50000 THEN 'Medium' ELSE 'Low' END as salary_band FROM employees.

Follow-up Questions:

- Can you use CASE in an ORDER BY clause?
- What happens if no WHEN condition matches and there is no ELSE?

What Interviewer Looks For: Practical use of CASE for data transformation and conditional logic in queries.

Common Mistakes: Forgetting the END keyword. Not knowing NULL is returned when no condition matches without an ELSE.

Q37: What are string functions in PostgreSQL?

Difficulty: Beginner **Category:** Functions

Answer: PostgreSQL provides many string functions: LENGTH(str) returns string length; UPPER(str) and LOWER(str) change case; TRIM(str), LTRIM(str), RTRIM(str) remove whitespace; SUBSTRING(str FROM start FOR len) extracts a substring; CONCAT(str1, str2) or the || operator concatenates strings; REPLACE(str, from, to) replaces substrings; POSITION(substr IN str) finds position of substring; LEFT(str, n) and RIGHT(str, n) return n characters from start/end; LPAD and RPAD pad strings; SPLIT_PART(str, delimiter, n) splits and returns the nth part; REGEXP_REPLACE and REGEXP_MATCH for regex operations. String functions are case-sensitive unless ILIKE or LOWER/UPPER is used.

Follow-up Questions:

- What is the difference between LIKE and ILIKE?
- How do you use regular expressions in PostgreSQL string functions?

What Interviewer Looks For: Familiarity with common string functions and PostgreSQL-specific ones like SPLIT_PART.

Common Mistakes: Not knowing ILIKE for case-insensitive matching. Using LIKE '%term%' for full-text search (use pg_trgm or full-text search instead).

Q38: What is the LIKE operator?

Difficulty: Beginner **Category:** Basic SQL

Answer: LIKE performs pattern matching on strings using wildcards. The % wildcard matches any sequence of characters (including zero). The _ wildcard matches exactly one character. Examples: WHERE name LIKE 'A%' finds names starting with A; WHERE name LIKE '%son' finds names ending with son; WHERE code LIKE 'A_C' matches A followed by any character followed by C. LIKE is case-sensitive in PostgreSQL. ILIKE is PostgreSQL's case-insensitive version. To use % or _ as literal characters, escape them with a backslash or use the ESCAPE clause. LIKE with a leading % (e.g., LIKE '%term') cannot use a B-tree index and requires a full table scan.

Follow-up Questions:

- How do you escape special characters in LIKE?
- What index type supports LIKE with a prefix?

What Interviewer Looks For: Pattern matching with %, _ case sensitivity, and the indexing limitation of leading wildcards.

Common Mistakes: Using LIKE when = would suffice. Not knowing that leading % defeats index usage.

Q39: What are date and time functions in PostgreSQL?

Difficulty: Beginner **Category:** Functions

Answer: PostgreSQL has extensive date/time functions: NOW() returns the current timestamp with timezone; CURRENT_DATE returns today's date; CURRENT_TIME returns the current time; AGE(timestamp) calculates interval from timestamp to now; EXTRACT(field FROM timestamp) extracts year, month, day, hour, etc.; DATE_TRUNC(field, timestamp) truncates to specified precision; DATE_PART(field, timestamp) is similar to EXTRACT; INTERVAL arithmetic allows adding/subtracting time periods (NOW() - INTERVAL '7 days'); TO_CHAR(timestamp, format) formats dates as strings; TO_TIMESTAMP(string, format) parses strings to timestamps. TIMESTAMPTZ stores timestamps with timezone, which is recommended for most use cases.

Follow-up Questions:

- What is the difference between `TIMESTAMP` and `TIMESTAMPTZ`?
- How do you calculate the age between two dates?

What Interviewer Looks For: Practical date manipulation including `EXTRACT`, `DATE_TRUNC`, and `INTERVAL` arithmetic.

Common Mistakes: Using `TIMESTAMP` instead of `TIMESTAMPTZ` in applications with multiple timezones. Not knowing `DATE_TRUNC` for grouping by month/year.

Q40: What is the difference between `TIMESTAMP` and `TIMESTAMPTZ`?

Difficulty: Beginner **Category:** Data Types

Answer: `TIMESTAMP` (or `TIMESTAMP WITHOUT TIME ZONE`) stores a date and time without any timezone information. `TIMESTAMPTZ` (or `TIMESTAMP WITH TIME ZONE`) stores a date and time with timezone awareness — PostgreSQL converts the input to UTC for storage and converts it back to the session's timezone when displaying. `TIMESTAMPTZ` is almost always the right choice for applications dealing with users in different timezones or running on servers in different regions. `TIMESTAMP` can lead to subtle bugs when server timezone changes or when data is migrated. The stored bytes are the same size; the difference is only in interpretation.

Follow-up Questions:

- How does PostgreSQL store `TIMESTAMPTZ` internally?
- What timezone does PostgreSQL convert `TIMESTAMPTZ` to when displaying?

What Interviewer Looks For: Understanding that `TIMESTAMPTZ` stores UTC and converts on display, and why it's preferred.

Common Mistakes: Using `TIMESTAMP` and then having timezone-related bugs. Not knowing that `TIMESTAMPTZ` converts to the session timezone for display.

Q41: How do you add a column to an existing table?

Difficulty: Beginner **Category:** DDL

Answer: Use `ALTER TABLE ... ADD COLUMN`: `ALTER TABLE employees ADD COLUMN phone VARCHAR(20)`. You can include constraints: `ALTER TABLE employees ADD COLUMN phone VARCHAR(20) NOT NULL DEFAULT 'N/A'`. In PostgreSQL, adding a nullable column with no default is fast (just a catalog change). Adding a `NOT NULL` column with a default used to require a full table rewrite in older versions, but since PostgreSQL 11, if the default is not volatile (not `NOW()` for example), it is stored in the catalog and the table is not rewritten. Adding a column with no default that is `NOT NULL` requires a `DEFAULT` or will fail if the table has existing rows.

Follow-up Questions:

- How do you add a `NOT NULL` column to a large table without downtime?
- What changed in PostgreSQL 11 regarding adding columns with defaults?

What Interviewer Looks For: Knowing the PostgreSQL 11 improvement for non-volatile defaults and the safe pattern for large tables.

Common Mistakes: Not knowing that adding a `NOT NULL` column with a volatile default (like `NOW()`) still requires a table rewrite.

Q42: How do you rename a column or table?

Difficulty: Beginner **Category:** DDL

Answer: To rename a column: `ALTER TABLE table_name RENAME COLUMN old_name TO new_name`. To rename a table: `ALTER TABLE old_name RENAME TO new_name`. These operations are DDL but are transactional in PostgreSQL. Renaming takes a brief ACCESS EXCLUSIVE lock on the table, which blocks all concurrent access. This is usually fast since it is just a catalog update. You should be careful when renaming columns or tables because any views, functions, or application code referencing the old names will break. To rename a schema: `ALTER SCHEMA old_name RENAME TO new_name`.

Follow-up Questions:

- Does renaming a table break views that reference it?
- What lock does RENAME acquire?

What Interviewer Looks For: Knowing the syntax and being aware of the implications for dependent objects.

Common Mistakes: Not checking for dependent views or functions before renaming. Not knowing that this takes an ACCESS EXCLUSIVE lock.

Q43: How do you DROP a table?

Difficulty: Beginner **Category:** DDL

Answer: `DROP TABLE` removes a table and all its data permanently: `DROP TABLE table_name`. `DROP TABLE IF EXISTS` prevents an error if the table doesn't exist. If other tables have foreign keys referencing the table, `DROP TABLE` will fail unless you use `CASCADE`: `DROP TABLE table_name CASCADE` — this also drops the dependent foreign key constraints (and potentially dependent views). `DROP TABLE` requires the table owner or superuser privileges. It is irreversible — always take a backup before dropping production tables. In PostgreSQL, `DROP TABLE` is transactional and can be rolled back within a transaction.

Follow-up Questions:

- What is the difference between `DROP TABLE` and `TRUNCATE`?
- How do you drop multiple tables at once?

What Interviewer Looks For: Awareness of `CASCADE`, the transactional nature of DDL in PostgreSQL, and the irreversibility.

Common Mistakes: Using `DROP TABLE` in production without a backup. Not knowing that `CASCADE` drops dependent constraints.

Q44: What is a subquery?

Difficulty: Beginner **Category:** Subqueries

Answer: A subquery is a `SELECT` statement nested inside another SQL statement. It can appear in the `SELECT`, `FROM`, or `WHERE` clause. Subqueries in `WHERE`: `SELECT * FROM employees WHERE department_id IN (SELECT id FROM departments WHERE name = 'Engineering')`. Scalar subqueries in `SELECT` return a single value: `SELECT name, (SELECT COUNT(*) FROM orders WHERE customer_id = c.id) as order_count FROM customers c`. Subqueries in `FROM` (derived tables): `SELECT avg_salary FROM (SELECT department, AVG(salary) as avg_salary`

FROM employees GROUP BY department) t WHERE avg_salary > 80000. Correlated subqueries reference columns from the outer query and run once per outer row.

Follow-up Questions:

- What is a correlated subquery?
- When would you use a subquery vs a JOIN?

What Interviewer Looks For: Understanding of the different positions subqueries can appear and the difference between correlated and non-correlated.

Common Mistakes: Using correlated subqueries where a JOIN would be more efficient. Not knowing that correlated subqueries execute once per outer row.

Q45: What is the IN operator?

Difficulty: Beginner **Category:** Basic SQL

Answer: The IN operator checks whether a value matches any value in a list or subquery. Example with list: WHERE country IN ('USA', 'Canada', 'UK'). Example with subquery: WHERE department_id IN (SELECT id FROM departments WHERE active = true). NOT IN does the opposite — matches rows where the value is NOT in the list. Caution: NOT IN with a subquery that returns any NULL values will return no rows at all because NOT IN (NULL) is NULL, not FALSE. For this reason, NOT EXISTS or a LEFT JOIN ... IS NULL is often preferred over NOT IN with subqueries.

Follow-up Questions:

- Why should you avoid NOT IN with a subquery that might return NULLs?
- What is the difference between IN and EXISTS?

What Interviewer Looks For: The NULL trap with NOT IN — this is a classic interview gotcha that trips up many candidates.

Common Mistakes: Using NOT IN with a subquery that can return NULLs. Not knowing that NULL in the IN list causes unexpected behavior.

Q46: What is the EXISTS operator?

Difficulty: Beginner **Category:** Subqueries

Answer: EXISTS tests whether a subquery returns any rows. It returns TRUE if the subquery returns at least one row, FALSE otherwise. The subquery's actual column values don't matter — only whether rows exist. Example: SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id) finds customers who have at least one order. NOT EXISTS finds customers with no orders. EXISTS is often more efficient than IN for large datasets because it short-circuits — it stops as soon as it finds the first matching row. EXISTS handles NULLs correctly unlike NOT IN.

Follow-up Questions:

- What is the difference between EXISTS and IN?
- Why is NOT EXISTS safer than NOT IN?

What Interviewer Looks For: Understanding of short-circuit evaluation and the NULL safety advantage over NOT IN.

Common Mistakes: Writing `SELECT *` in the `EXISTS` subquery (convention is `SELECT 1` since columns don't matter). Using `IN` when `EXISTS` would be more appropriate.

Q47: What is the BETWEEN operator?

Difficulty: Beginner **Category:** Basic SQL

Answer: `BETWEEN` checks if a value falls within an inclusive range: `WHERE salary BETWEEN 50000 AND 80000` is equivalent to `WHERE salary >= 50000 AND salary <= 80000`. Both endpoints are inclusive. It works with numbers, dates, and strings (alphabetical range). `NOT BETWEEN` excludes the range. For dates: `WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31'`. Be careful with `BETWEEN` and `TIMESTAMP` — `BETWEEN '2024-01-01' AND '2024-12-31'` on a timestamp column will miss times on December 31st after midnight, so it's better to use `>=` and `<` with timestamps.

Follow-up Questions:

- Is `BETWEEN` inclusive or exclusive?
- What is the issue with using `BETWEEN` on `TIMESTAMP` columns?

What Interviewer Looks For: Knowing `BETWEEN` is inclusive on both ends and the timestamp gotcha.

Common Mistakes: Thinking `BETWEEN` is exclusive. Using `BETWEEN` on timestamps and missing the end-of-day rows.

Q48: How do you use aliases in SQL?

Difficulty: Beginner **Category:** Basic SQL

Answer: Aliases give temporary names to columns or tables in a query. Column alias: `SELECT salary * 12 AS annual_salary FROM employees` or `SELECT salary * 12 annual_salary (AS is optional)`. Table alias: `SELECT e.name FROM employees e` or `SELECT e.name FROM employees AS e`. Aliases are required for subqueries used in `FROM`: `SELECT * FROM (SELECT ...) AS subq`. Column aliases defined in `SELECT` can be used in `ORDER BY` but not `WHERE` or `HAVING` (because of execution order). Table aliases are especially useful for self-joins and for shortening long table names. Aliases are local to the query and do not change database objects.

Follow-up Questions:

- Can you use a column alias in `WHERE`?
- Why are aliases required for subqueries in `FROM`?

What Interviewer Looks For: Understanding execution order constraints on alias usage and when aliases are mandatory.

Common Mistakes: Trying to use a `SELECT` alias in `WHERE`. Forgetting that subquery aliases are required.

Q49: What is the difference between = and LIKE in SQL?

Difficulty: Beginner **Category:** Basic SQL

Answer: The `=` operator performs exact equality matching. The `LIKE` operator performs pattern matching using wildcards (`%` for any sequence, `_` for any single character). `=` is faster because it can use a standard B-tree index efficiently, while `LIKE '%pattern%'` with a leading wildcard cannot use a B-tree index. For exact matches, always use `=`. For starts-with patterns (`LIKE 'prefix%'`), a B-tree index can be used. `LIKE` is case-

sensitive in PostgreSQL; use ILIKE for case-insensitive pattern matching. The ~ operator and regular expression matching are available for more complex patterns.

Follow-up Questions:

- What is the ~ operator in PostgreSQL?
- How can you make LIKE case-insensitive?

What Interviewer Looks For: Performance implications of different operators and when to choose each.

Common Mistakes: Using LIKE for exact matches instead of =. Using LIKE with a leading wildcard and expecting it to be index-supported.

Q50: What is the CONCAT function and || operator?

Difficulty: Beginner **Category:** Functions

Answer: Both CONCAT() and || concatenate strings. The key difference is NULL handling: the || operator returns NULL if any operand is NULL, while CONCAT() treats NULL as an empty string. Example: 'Hello' || ' ' || 'World' returns 'Hello World'. CONCAT('Hello', NULL, 'World') returns 'HelloWorld'. CONCAT_WS(separator, str1, str2, ...) concatenates with a separator, skipping NULLs. In PostgreSQL, || can also be used for array and jsonb concatenation. For building strings with mixed types, you may need to cast: 'Employee #' || employee_id::text. The || operator is SQL standard while CONCAT is a function available in most databases.

Follow-up Questions:

- How does CONCAT handle NULL values compared to ||?
- What is CONCAT_WS?

What Interviewer Looks For: NULL handling difference between || and CONCAT, and awareness of CONCAT_WS.

Common Mistakes: Using || with NULLs and getting unexpected NULL results. Not knowing CONCAT_WS for building delimited strings.

Q51: What is a stored procedure in PostgreSQL?

Difficulty: Beginner **Category:** Stored Procedures

Answer: A stored procedure (created with CREATE PROCEDURE) is a named, stored set of SQL and procedural code that can be called with CALL statement. In PostgreSQL 11+, procedures support transaction control (COMMIT, ROLLBACK within the procedure). Functions (CREATE FUNCTION) are similar but return a value and cannot control transactions. Stored procedures are written in PL/pgSQL (or other languages like PL/Python). They accept parameters and can perform complex logic. Benefits include code reuse, reduced network round-trips, security (grant execute without table access), and performance. Example: CALL update_employee_salary(emp_id := 1, new_salary := 75000).

Follow-up Questions:

- What is the difference between a stored procedure and a function in PostgreSQL?
- What languages can you use to write PostgreSQL functions?

What Interviewer Looks For: Understanding the procedure vs function distinction, especially transaction control.

Common Mistakes: Using CALL for functions (use SELECT instead). Not knowing the procedure vs function distinction regarding transaction control.

Q52: What is PL/pgSQL?

Difficulty: Beginner **Category:** Stored Procedures

Answer: PL/pgSQL (Procedural Language/PostgreSQL) is PostgreSQL's default procedural language for writing stored functions and procedures. It extends SQL with control structures like IF/ELSIF/ELSE, loops (LOOP, WHILE, FOR), variables, exception handling (BEGIN/EXCEPTION/END blocks), and cursors. It is tightly integrated with SQL and can execute arbitrary SQL statements. Variables are declared in a DECLARE block. Example structure: `CREATE FUNCTION func() RETURNS type AS $$ DECLARE v_count INTEGER; BEGIN SELECT COUNT(*) INTO v_count FROM table; RETURN v_count; END; $$ LANGUAGE plpgsql`. PL/pgSQL compiles to an internal form and is cached, making repeated calls efficient.

Follow-up Questions:

- What other procedural languages does PostgreSQL support?
- How do you handle exceptions in PL/pgSQL?

What Interviewer Looks For: Basic structure of a PL/pgSQL function including DECLARE, BEGIN, END, and LANGUAGE clause.

Common Mistakes: Forgetting the \$\$ delimiters. Not declaring variables before the BEGIN block.

Q53: What is normalization in databases?

Difficulty: Beginner **Category:** Database Design

Answer: Normalization is the process of organizing a database to reduce data redundancy and improve data integrity. It involves decomposing tables following normal forms. First Normal Form (1NF): each column contains atomic values, no repeating groups. Second Normal Form (2NF): 1NF + every non-key attribute is fully dependent on the entire primary key (no partial dependencies). Third Normal Form (3NF): 2NF + no transitive dependencies (non-key attributes depend only on the key). Boyce-Codd Normal Form (BCNF) is a stronger version of 3NF. Normalization reduces anomalies (insertion, update, deletion) but can require more JOINS in queries. Denormalization is intentional violation for performance.

Follow-up Questions:

- What is the difference between 2NF and 3NF?
- When would you intentionally denormalize?

What Interviewer Looks For: Understanding of the normal forms, the anomalies they prevent, and the tradeoff with query performance.

Common Mistakes: Memorizing definitions without understanding why normalization matters. Not knowing when denormalization is appropriate.

Q54: What is denormalization and when would you use it?

Difficulty: Beginner **Category:** Database Design

Answer: Denormalization is the process of intentionally introducing redundancy into a database design to improve read performance. Instead of always joining multiple normalized tables, you store redundant data

together to speed up queries. Common techniques include storing calculated values, duplicating columns across tables, or combining tables that would normally be split. Use denormalization when: read performance is critical and the data is read far more than written; JOINS are expensive because tables are very large; real-time analytics require pre-computed aggregates. The tradeoffs are: increased storage, risk of data inconsistency, and more complex writes. OLAP systems are typically more denormalized than OLTP systems.

Follow-up Questions:

- What is the difference between OLTP and OLAP schema design?
- How do materialized views help with denormalization?

What Interviewer Looks For: Understanding the OLAP vs OLTP tradeoff and concrete examples of when denormalization is justified.

Common Mistakes: Thinking denormalization is always bad. Not being able to give concrete examples of when to use it.

Q55: What is referential integrity?

Difficulty: Beginner **Category:** Constraints

Answer: Referential integrity ensures that relationships between tables remain consistent. Specifically, it ensures that foreign key values in a child table always reference existing values in the parent table. For example, if an orders table has a customer_id foreign key, referential integrity prevents creating an order for a customer that doesn't exist. PostgreSQL enforces referential integrity through foreign key constraints. When a parent row is deleted, you can CASCADE the deletion, SET NULL in child rows, RESTRICT the deletion, or SET DEFAULT. Referential integrity is critical for data quality and prevents orphaned records.

Follow-up Questions:

- What happens when you try to delete a parent row referenced by a child?
- How does DEFERRABLE foreign key constraint work?

What Interviewer Looks For: Understanding of how FK constraints enforce referential integrity and the cascade options.

Common Mistakes: Not knowing the different ON DELETE actions. Not knowing that referential integrity checks can be deferred until end of transaction with DEFERRABLE.

Q56: What is the difference between a clustered and non-clustered index?

Difficulty: Beginner **Category:** Indexes

Answer: In most databases, a clustered index determines the physical order of data storage — the table rows are stored in the same order as the index. A non-clustered index is a separate structure that contains the index key and a pointer to the row. PostgreSQL does not have traditional clustered indexes in the SQL Server sense. In PostgreSQL, you can use the CLUSTER command to physically reorder a table according to an index, but this is a one-time operation that doesn't stay synchronized. All PostgreSQL indexes (B-tree, etc.) are logically similar to "non-clustered" and store a pointer (ctid) back to the heap. BRIN indexes are an exception — they work best on naturally ordered data.

Follow-up Questions:

- What is the CLUSTER command in PostgreSQL?
- What is a heap in PostgreSQL?

What Interviewer Looks For: Understanding that PostgreSQL's model differs from SQL Server's, and knowing what CLUSTER does.

Common Mistakes: Applying SQL Server's clustered/non-clustered concepts directly to PostgreSQL. Not knowing that CLUSTER is a one-time reorganization.

Q57: What is a composite index?

Difficulty: Beginner **Category:** Indexes

Answer: A composite index (multi-column index) is an index on multiple columns. `CREATE INDEX idx_name ON table (col1, col2, col3)`. The order of columns matters — a composite index on (col1, col2) can be used for queries filtering on col1 alone or col1 AND col2, but generally not col2 alone. This is the "leading column" rule. Composite indexes are useful when queries frequently filter on the same set of columns together. They can also cover queries — if all columns needed by a query are in the index, PostgreSQL can satisfy the query from the index alone without reading the table (index-only scan).

Follow-up Questions:

- Does column order matter in a composite index?
- What is an index-only scan?

What Interviewer Looks For: Understanding of the leading column rule and covering indexes.

Common Mistakes: Creating indexes without considering column order. Not knowing about index-only scans.

Q58: What is the EXPLAIN command?

Difficulty: Beginner **Category:** Performance

Answer: EXPLAIN shows the query execution plan that PostgreSQL's query planner will use without actually executing the query. EXPLAIN ANALYZE actually runs the query and shows both the estimated and actual row counts, costs, and execution times. The plan is a tree of nodes showing operations like Seq Scan, Index Scan, Hash Join, Sort, etc. Key metrics: `cost=(startup_cost..total_cost)`, `rows=estimated_rows`, `width=estimated_row_size`. The most important use of EXPLAIN ANALYZE is identifying: sequential scans on large tables that should use indexes; nested loop joins that should be hash joins; and incorrect row estimates.

Follow-up Questions:

- What is the difference between EXPLAIN and EXPLAIN ANALYZE?
- What does the cost in an execution plan represent?

What Interviewer Looks For: Basic understanding of execution plans and the difference between EXPLAIN and EXPLAIN ANALYZE.

Common Mistakes: Using EXPLAIN ANALYZE on a DELETE/UPDATE without wrapping in a transaction to roll back. Not understanding that cost is an arbitrary unit, not actual time.

Q59: What is a database trigger?

Difficulty: Beginner **Category:** Database Objects

Answer: A trigger is a stored function that automatically executes in response to specific events on a table or view: INSERT, UPDATE, DELETE, or TRUNCATE. Triggers can fire BEFORE or AFTER the event, or INSTEAD OF (for views). They can fire FOR EACH ROW (once per modified row) or FOR EACH STATEMENT (once per SQL statement). In PostgreSQL, triggers call trigger functions written in PL/pgSQL or another language. Trigger functions have access to OLD and NEW row variables (OLD for before-change row, NEW for after). Common uses: audit logging, automatic timestamp updates, data validation, maintaining derived data.

Follow-up Questions:

- What is the difference between BEFORE and AFTER triggers?
- What are OLD and NEW in a trigger function?

What Interviewer Looks For: Understanding of trigger types, the OLD/NEW special variables, and common use cases.

Common Mistakes: Not knowing the difference between ROW-level and STATEMENT-level triggers. Overusing triggers and creating hard-to-debug side effects.

Q60: What is a database connection pool?

Difficulty: Beginner **Category:** Performance

Answer: A connection pool is a cache of database connections that are reused rather than opened and closed for each request. Opening a PostgreSQL connection is expensive — it involves authentication, process forking, and memory allocation. Without pooling, a web application would open and close a connection for every request, wasting time and resources. Connection pools maintain a set of open connections ready to be reused. Popular PostgreSQL connection poolers include PgBouncer and Pgpool-II. Application-level pools are provided by libraries like psycopg2, SQLAlchemy, or JDBC. Key settings: pool_size (number of connections), max_overflow, pool_timeout.

Follow-up Questions:

- What is PgBouncer?
- What are the different modes of connection pooling?

What Interviewer Looks For: Understanding of why pooling is needed and awareness of PostgreSQL-specific tools like PgBouncer.

Common Mistakes: Not knowing that PostgreSQL forks a process per connection. Not knowing about PgBouncer's transaction vs session pooling modes.

Q61: What is the difference between INNER JOIN and OUTER JOIN?

Difficulty: Beginner **Category:** Joins

Answer: An INNER JOIN returns only rows that have matching values in both tables. An OUTER JOIN returns rows even when there is no match in one of the tables, filling the unmatched side with NULLs. The three outer join types are LEFT OUTER JOIN (all rows from left, NULLs for unmatched right), RIGHT OUTER JOIN (all rows from right, NULLs for unmatched left), and FULL OUTER JOIN (all rows from both, NULLs where unmatched). For example, getting all employees even if they have no manager would require a LEFT JOIN. Getting all potential pairs even where data is missing requires FULL OUTER JOIN.

Follow-up Questions:

- When would you use a FULL OUTER JOIN?
- How do you simulate a FULL OUTER JOIN in databases that don't support it?

What Interviewer Looks For: Clear explanation of which rows are included in each join type and when each is appropriate.

Common Mistakes: Not knowing FULL OUTER JOIN exists. Confusing which table is "left" and which is "right".

Q62: How do you perform a SELF JOIN?

Difficulty: Beginner **Category:** Joins

Answer: A SELF JOIN joins a table to itself, requiring at least two aliases for the same table. It is useful for hierarchical data or comparing rows within the same table. Classic example — finding employees and their managers from the same employees table: `SELECT e.name AS employee, m.name AS manager FROM employees e LEFT JOIN employees m ON e.manager_id = m.id`. Another example: finding pairs of cities within 100 miles. The LEFT JOIN is used so employees with no manager (NULL manager_id) also appear in the result. SELF JOINS are functionally the same as regular JOINS, just against the same table twice.

Follow-up Questions:

- What is the difference between a SELF JOIN and a recursive CTE for hierarchical data?
- How do you avoid getting both (A,B) and (B,A) pairs in a SELF JOIN?

What Interviewer Looks For: Ability to write a self-join with proper aliases and understanding of when to use LEFT vs INNER.

Common Mistakes: Forgetting to alias both instances of the table. Using INNER JOIN and losing employees without managers.

Q63: What is the CAST function and :: notation?

Difficulty: Beginner **Category:** Data Types

Answer: CAST converts a value from one data type to another. Standard SQL syntax: `CAST(value AS type)`. PostgreSQL-specific shorthand: `value::type`. Examples: `'42'::INTEGER`, `NOW()::DATE`, `3.14::TEXT`, `'2024-01-01'::TIMESTAMP`. Explicit casts are needed when PostgreSQL cannot implicitly convert between types. Common use cases: converting strings to numbers for arithmetic, converting timestamps to dates, converting integers to text for concatenation. If the cast fails (e.g., casting 'abc' to INTEGER), PostgreSQL raises an error. TRY_CAST does not exist in PostgreSQL, but you can use a custom function or regexp to validate before casting.

Follow-up Questions:

- What is the difference between implicit and explicit casting?
- How do you safely cast a string to a number?

What Interviewer Looks For: Familiarity with both CAST syntax and the PostgreSQL :: shorthand.

Common Mistakes: Not knowing the :: shorthand. Not handling cast failures gracefully.

Q64: What is the ROUND function?

Difficulty: Beginner **Category:** Functions

Answer: ROUND(value, decimals) rounds a numeric value to a specified number of decimal places.

ROUND(3.14159, 2) returns 3.14. ROUND(3.14159, 0) returns 3. ROUND(3.5) returns 4. With negative decimal places, ROUND rounds to the left of the decimal: ROUND(123.456, -1) returns 120. PostgreSQL uses "round half to even" (banker's rounding) for NUMERIC types — ROUND(2.5) returns 2, ROUND(3.5) returns 4. For DOUBLE PRECISION, it uses traditional round-half-up. Related functions: CEIL/CEILING (round up), FLOOR (round down), TRUNC (truncate toward zero).

Follow-up Questions:

- What is the difference between ROUND, CEIL, FLOOR, and TRUNC?
- What is banker's rounding?

What Interviewer Looks For: Knowing the difference between ROUND, CEIL, FLOOR, TRUNC, and the banker's rounding behavior.

Common Mistakes: Not knowing that ROUND for NUMERIC uses banker's rounding. Confusing TRUNC (toward zero) with FLOOR (toward negative infinity) for negative numbers.

Q65: How do you use the COALESCE function with multiple arguments?

Difficulty: Beginner **Category:** Functions

Answer: COALESCE evaluates its arguments left to right and returns the first non-NULL value. With multiple arguments: COALESCE(a, b, c, d) returns the first of a, b, c, d that is not NULL. If all are NULL, it returns NULL. Example: SELECT COALESCE(nickname, first_name, 'Unknown') FROM users — returns nickname if set, otherwise first_name if set, otherwise 'Unknown'. This is useful for building fallback chains. COALESCE short-circuits: if the first argument is non-NULL, the remaining arguments are not evaluated (important if they are expensive expressions or function calls).

Follow-up Questions:

- Does COALESCE evaluate all its arguments?
- How would you use COALESCE in an UPDATE statement?

What Interviewer Looks For: Short-circuit behavior and practical multi-fallback patterns.

Common Mistakes: Not knowing about short-circuit evaluation. Using nested CASE WHEN instead of cleaner multi-argument COALESCE.

Q66: What is the difference between a function and an operator in PostgreSQL?

Difficulty: Beginner **Category:** Functions

Answer: Functions are called with function syntax: func_name(arg1, arg2). Operators are special symbols that appear between operands: a + b, a = b, a || b. In PostgreSQL, operators are actually implemented as functions — you can define custom operators backed by functions. Built-in operators include arithmetic (+, -, *, /), comparison (=, <, <=, >, >=), logical (AND, OR, NOT), string (||), and PostgreSQL-specific ones (JSONB access ->, ->>, #>, array contains @>, etc.). The difference is syntactic — operators enable infix notation. Some operations like ILIKE, BETWEEN, IN, IS NULL are technically operators in SQL.

Follow-up Questions:

- Can you create custom operators in PostgreSQL?
- What are the JSONB operators?

What Interviewer Looks For: Understanding that operators are syntactic sugar over functions and knowing the common PostgreSQL operators.

Common Mistakes: Not knowing PostgreSQL-specific operators for JSONB and arrays.

Q67: What is a temporary table?

Difficulty: Beginner **Category:** Database Objects

Answer: A temporary table is a table that exists only for the duration of a database session (or transaction, with ON COMMIT DROP). Created with CREATE TEMP TABLE or CREATE TEMPORARY TABLE. Temporary tables are only visible to the session that created them — other sessions cannot see or access them. They are automatically dropped when the session ends. Useful for: storing intermediate results of complex calculations, staging data before processing, and avoiding repeated subquery execution. Each session gets its own copy of the temp table even if they have the same name. Temp tables can have indexes, constraints, and work like regular tables.

Follow-up Questions:

- How do temporary tables differ from CTEs?
- What is ON COMMIT DROP?

What Interviewer Looks For: Session-scope vs transaction-scope, visibility isolation between sessions, and use cases.

Common Mistakes: Thinking temp tables are shared between sessions. Not knowing ON COMMIT DROP for transaction-scoped temp tables.

Q68: What is a CTE (Common Table Expression)?

Difficulty: Beginner **Category:** Subqueries

Answer: A CTE (Common Table Expression) is a named temporary result set defined using the WITH clause, which can be referenced within the main SELECT, INSERT, UPDATE, or DELETE. Syntax: WITH cte_name AS (SELECT ...) SELECT * FROM cte_name. CTEs improve query readability by breaking complex queries into named building blocks. Multiple CTEs can be defined and can reference each other. Unlike subqueries, CTEs can be referenced multiple times in the same query. PostgreSQL also supports recursive CTEs (WITH RECURSIVE) for hierarchical queries. In PostgreSQL, CTEs were historically optimization fences (preventing the planner from pushing predicates through), but since PostgreSQL 12, they can be inlined by default.

Follow-up Questions:

- What is a recursive CTE?
- What changed in PostgreSQL 12 regarding CTE optimization?

What Interviewer Looks For: Syntax, multiple CTE references, recursive CTEs, and the PostgreSQL 12 change.

Common Mistakes: Not knowing about recursive CTEs. Not knowing the PostgreSQL 12 change that made CTEs inlinable.

Q69: How do you copy data from a CSV file into PostgreSQL?

Difficulty: Beginner **Category:** Data Loading

Answer: PostgreSQL's COPY command loads data from files: COPY table_name FROM '/path/to/file.csv' DELIMITER ',' CSV HEADER. The HEADER option skips the first row. You can specify specific columns: COPY table_name (col1, col2) FROM '/path/to/file.csv' CSV. The server-side COPY requires the file to be on the PostgreSQL server. The client-side \COPY command (in psql) reads files from the client machine. COPY TO exports data to a file. For large imports, COPY is much faster than repeated INSERTs. You can also COPY from stdin: COPY table FROM stdin. In cloud environments, COPY can read from programs with COPY FROM PROGRAM.

Follow-up Questions:

- What is the difference between COPY and \copy?
- How can you handle errors during a COPY import?

What Interviewer Looks For: Knowing both server-side COPY and client-side \copy, and performance advantage over INSERTs.

Common Mistakes: Trying to use server-side COPY with a client-side file path. Not knowing about CSV HEADER option.

Q70: What is the psql command-line tool?

Difficulty: Beginner **Category:** Tools

Answer: psql is PostgreSQL's interactive command-line terminal for executing SQL commands and administrative tasks. You connect with: psql -U username -d database_name -h host. Useful psql meta-commands (start with \): \l or \list — list databases; \c dbname — connect to database; \dt — list tables; \d tablename — describe table; \df — list functions; \i file.sql — execute SQL file; \timing — toggle query timing; \e — open editor; \copy — client-side file import/export; \q — quit. psql supports tab completion, command history, and variable substitution. It is the primary tool for interactive PostgreSQL administration.

Follow-up Questions:

- How do you run a SQL file using psql?
- What is the difference between COPY and \copy in psql?

What Interviewer Looks For: Familiarity with common psql meta-commands, especially \d for inspecting objects.

Common Mistakes: Not knowing psql meta-commands. Forgetting that psql commands start with \ and are different from SQL.

Q71: What is pg_dump and how is it used?

Difficulty: Beginner **Category:** Administration

Answer: pg_dump is a utility for backing up a PostgreSQL database. It produces a script file or archive file with SQL commands to recreate the database. Basic usage: pg_dump -U username -d dbname -f backup.sql. For compressed archives: pg_dump -Fc -U username -d dbname -f backup.dump. pg_dump takes a consistent snapshot — it is safe to use on a live database. To restore: psql < backup.sql or pg_restore -d dbname

backup.dump. pg_dumpall backs up all databases plus roles and tablespaces. Common options: -t tablename (specific table), -s (schema only), -a (data only), --no-owner, --no-privileges.

Follow-up Questions:

- What is the difference between pg_dump and pg_dumpall?
- What is the -Fc format and why use it over plain SQL?

What Interviewer Looks For: Basic backup/restore workflow and knowing the custom format (-Fc) for parallel restore.

Common Mistakes: Not knowing that pg_restore is needed for custom format dumps. Not knowing about pg_dumpall for global objects.

Q72: What are PostgreSQL roles and users?

Difficulty: Beginner **Category:** Administration

Answer: In PostgreSQL, users and roles are the same concept — CREATE USER is equivalent to CREATE ROLE WITH LOGIN. A role can be a user (has LOGIN privilege), a group (used to aggregate permissions), or both. Roles can own database objects and have privileges. Key privilege management: GRANT privilege ON object TO role; REVOKE privilege ON object FROM role. Common privileges: SELECT, INSERT, UPDATE, DELETE, EXECUTE, CONNECT, CREATEDB, CREATEROLE, SUPERUSER. Role membership allows inheriting privileges: GRANT group_role TO user_role. The superuser bypasses all permission checks. Best practice: create a dedicated role with minimum required privileges for each application.

Follow-up Questions:

- What is the difference between a role and a user in PostgreSQL?
- What is the principle of least privilege?

What Interviewer Looks For: Understanding that users and roles are the same, privilege management, and security best practices.

Common Mistakes: Not knowing CREATE USER and CREATE ROLE are the same with different defaults. Not understanding role inheritance.

Q73: What is VACUUM in PostgreSQL?

Difficulty: Beginner **Category:** Maintenance

Answer: VACUUM reclaims storage occupied by dead tuples — rows that have been deleted or updated but are still physically present in the table because other transactions might still need to see the old version (MVCC). Without regular VACUUM, tables grow indefinitely with dead tuples, wasting disk space and slowing down queries. VACUUM (without FULL) marks space as reusable but doesn't return it to the OS. VACUUM FULL compacts the table and returns space to the OS but requires an exclusive lock. AUTOVACUUM is a background process that runs VACUUM automatically. VACUUM also updates statistics used by the query planner and prevents transaction ID wraparound.

Follow-up Questions:

- What is the difference between VACUUM and VACUUM FULL?
- What is autovacuum?

What Interviewer Looks For: Understanding of dead tuples from MVCC, autovacuum, and the difference between regular and FULL vacuum.

Common Mistakes: Not knowing WHY dead tuples exist (MVCC). Recommending VACUUM FULL for regular maintenance (should only be used rarely).

Q74: What is MVCC in PostgreSQL?

Difficulty: Beginner **Category:** Concurrency

Answer: MVCC (Multi-Version Concurrency Control) is PostgreSQL's mechanism for allowing concurrent transactions without locking readers. Instead of locking rows when they are read, PostgreSQL keeps multiple versions of each row. Readers see a consistent snapshot of the database at their transaction start time. Writers create new versions of rows rather than overwriting them. This means readers never block writers and writers never block readers. The old versions are retained as long as any transaction might still need them, then cleaned up by VACUUM. MVCC is key to PostgreSQL's excellent concurrent read/write performance.

Follow-up Questions:

- How does MVCC create dead tuples?
- What is a transaction snapshot in MVCC?

What Interviewer Looks For: Core concept: multiple row versions allow reads without blocking writes.

Common Mistakes: Thinking locking is the only concurrency mechanism. Not connecting MVCC to the need for VACUUM.

Q75: What is the difference between CHAR(10) and VARCHAR(10)?

Difficulty: Beginner **Category:** Data Types

Answer: CHAR(10) is fixed-length — it always stores exactly 10 characters, padding with spaces if the value is shorter. VARCHAR(10) is variable-length — it stores the actual string up to 10 characters without padding. For a value 'Hi', CHAR(10) stores 'Hi ' (8 spaces padding) while VARCHAR(10) stores 'Hi'. When comparing CHAR values, PostgreSQL strips trailing spaces (so CHAR(10) 'Hi' = CHAR(4) 'Hi'). In PostgreSQL, there is no performance advantage to using CHAR over VARCHAR. CHAR can cause subtle bugs due to space-padding behavior. In modern PostgreSQL, VARCHAR and TEXT are generally preferred over CHAR.

Follow-up Questions:

- What happens when you compare CHAR and VARCHAR values?
- When might CHAR be the right choice?

What Interviewer Looks For: The space padding behavior and comparison quirks of CHAR, and that there's no performance benefit in PostgreSQL.

Common Mistakes: Thinking CHAR is faster. Not knowing about the space-padding and comparison behavior.

Q76: What is a materialized view?

Difficulty: Beginner **Category:** Database Objects

Answer: A materialized view is like a regular view but stores the result of the query physically on disk. This makes reads very fast since the data is precomputed, but the data can become stale. You refresh it with

REFRESH MATERIALIZED VIEW view_name. Without CONCURRENTLY, refresh takes an exclusive lock. REFRESH MATERIALIZED VIEW CONCURRENTLY allows concurrent reads during refresh but requires a unique index on the view. Materialized views are excellent for expensive aggregation queries that are run frequently but can tolerate slightly stale data. Common use cases: dashboards, reporting, pre-computed analytics.

Follow-up Questions:

- When should you use a materialized view vs a regular view?
- What does CONCURRENTLY do in REFRESH MATERIALIZED VIEW?

What Interviewer Looks For: Understanding of the stale data tradeoff and the CONCURRENTLY option for non-blocking refresh.

Common Mistakes: Using a regular view when a materialized view would be much faster. Not knowing the CONCURRENTLY refresh option.

Q77: What is table inheritance in PostgreSQL?

Difficulty: Beginner **Category:** Database Objects

Answer: PostgreSQL supports table inheritance where a child table inherits all columns from a parent table. Syntax: CREATE TABLE child_table () INHERITS (parent_table). The child table automatically gets all parent columns and can add its own. Queries on the parent table return rows from both parent and child tables. SELECT * FROM parent also returns child rows (use ONLY to query just the parent: SELECT * FROM ONLY parent). Table inheritance was used for partitioning before PostgreSQL 10's declarative partitioning. It is less commonly used now but has niche use cases for polymorphic data models. Constraints and indexes on the parent are NOT inherited.

Follow-up Questions:

- How does table inheritance relate to partitioning?
- What is the ONLY keyword?

What Interviewer Looks For: Understanding that queries on parent include child rows, and that the ONLY keyword restricts to the parent.

Common Mistakes: Not knowing that indexes and constraints are not inherited. Confusing inheritance with foreign keys.

Q78: What is the ANY and ALL operators?

Difficulty: Beginner **Category:** Basic SQL

Answer: ANY (or SOME) returns TRUE if the comparison is TRUE for at least one value in a subquery or array. ALL returns TRUE only if the comparison is TRUE for every value. Examples: WHERE salary > ANY (SELECT salary FROM managers) — true if salary is greater than at least one manager's salary. WHERE salary > ALL (SELECT salary FROM managers) — true only if salary is greater than ALL managers' salaries. = ANY is equivalent to IN. <> ALL is equivalent to NOT IN (but safer with NULLs). These operators are used with comparison operators: =, <, >, <=, >=, <>.

Follow-up Questions:

- How is = ANY different from IN?
- Why is <> ALL safer than NOT IN with subqueries?

What Interviewer Looks For: Understanding of ANY vs ALL semantics and the equivalence with IN/NOT IN.

Common Mistakes: Not knowing that <> ALL handles NULLs differently (and more correctly) than NOT IN.

Q79: What is a cross join and when would you use it?

Difficulty: Beginner **Category:** Joins

Answer: A CROSS JOIN produces the Cartesian product of two tables — every row from the first table is combined with every row from the second table. If table A has 10 rows and table B has 5 rows, the result has 50 rows. Syntax: SELECT * FROM A CROSS JOIN B or simply SELECT * FROM A, B (implicit cross join). Use cases: generating all combinations (e.g., all possible product-color combinations), creating a calendar table by crossing dates with hours, or generating test data. CROSS JOINS are rarely used in practice and can accidentally produce enormous result sets if used unintentionally (forgetting a WHERE clause in a multi-table query).

Follow-up Questions:

- What is the implicit cross join syntax?
- How many rows does a cross join produce?

What Interviewer Looks For: Knowing that CROSS JOIN is Cartesian product, its use cases, and the danger of accidental cross joins.

Common Mistakes: Accidentally creating cross joins by forgetting a JOIN condition. Not knowing that the comma syntax in FROM creates an implicit cross join.

Q80: What is the HAVING clause and when do you use it?

Difficulty: Beginner **Category:** Basic SQL

Answer: HAVING filters groups created by GROUP BY based on aggregate conditions. It is like WHERE but applied after grouping. Example: SELECT department, AVG(salary) FROM employees GROUP BY department HAVING AVG(salary) > 70000 returns only departments where average salary exceeds 70,000. You must use HAVING (not WHERE) for any condition involving an aggregate function like COUNT, SUM, AVG, MIN, MAX. HAVING can also be used without GROUP BY when applied to a single-group aggregate (e.g., HAVING COUNT(*) > 100). For non-aggregate conditions, it is more efficient to put them in WHERE to filter rows before grouping.

Follow-up Questions:

- Can you use column aliases in HAVING?
- Can you have HAVING without GROUP BY?

What Interviewer Looks For: Clear understanding of when HAVING is required vs WHERE, and performance best practice of filtering early.

Common Mistakes: Putting aggregate conditions in WHERE. Using HAVING for non-aggregate conditions (less efficient than WHERE).

Q81: What are the different join conditions you can use?

Difficulty: Beginner **Category:** Joins

Answer: Joins are typically written with ON clause specifying the condition: JOIN t2 ON t1.id = t2.t1_id. The USING clause is a shorthand when both tables have the same column name: JOIN t2 USING (id). NATURAL JOIN automatically joins on all columns with the same name — generally avoided because it is fragile (adding a column can change join behavior). You can join on non-equality conditions: JOIN t2 ON t1.salary BETWEEN t2.low AND t2.high or JOIN t2 ON t1.timestamp < t2.end_date. Multiple conditions: ON t1.a = t2.a AND t1.b = t2.b. The join condition can include any boolean expression.

Follow-up Questions:

- Why is NATURAL JOIN generally avoided?
- What is the difference between ON and USING?

What Interviewer Looks For: Knowing ON vs USING, avoiding NATURAL JOIN, and non-equality joins.

Common Mistakes: Using NATURAL JOIN without understanding its fragility. Not knowing joins can use non-equality conditions.

Q82: What is a recursive query in PostgreSQL?

Difficulty: Beginner **Category:** Subqueries

Answer: A recursive query uses WITH RECURSIVE to iterate over hierarchical or graph-like data. Structure: WITH RECURSIVE cte AS (non_recursive_term UNION ALL recursive_term) SELECT * FROM cte. The non-recursive term (seed) provides the starting rows. The recursive term joins back to the CTE to expand the result. Example: traversing an employee hierarchy starting from the CEO down to all reports. PostgreSQL limits recursion depth with the search strategy (BREADTH FIRST/DEPTH FIRST) and cycle detection. Recursive CTEs are essential for hierarchical data like org charts, category trees, and network graphs.

Follow-up Questions:

- How do you prevent infinite loops in recursive CTEs?
- What is the difference between BREADTH FIRST and DEPTH FIRST search?

What Interviewer Looks For: Understanding of the seed + recursive term structure and cycle detection.

Common Mistakes: Creating infinite recursion by not having a proper base case. Not knowing about cycle detection clauses.

Q83: What is the difference between NUMERIC and FLOAT?

Difficulty: Beginner **Category:** Data Types

Answer: NUMERIC (or DECIMAL) is an exact numeric type that stores values with arbitrary precision. FLOAT (REAL or DOUBLE PRECISION) is an approximate type using IEEE 754 floating-point representation. For monetary calculations, always use NUMERIC because floating-point arithmetic has rounding errors: 0.1 + 0.2 in FLOAT does not equal exactly 0.3. NUMERIC is slower than FLOAT due to arbitrary precision arithmetic. FLOAT is faster and appropriate for scientific calculations where small rounding errors are acceptable. NUMERIC(10,2) means up to 10 total digits with 2 after the decimal point. For money in PostgreSQL, NUMERIC(19,4) is a common choice.

Follow-up Questions:

- Why should you never use FLOAT for monetary values?
- What is the MONEY type in PostgreSQL?

What Interviewer Looks For: The critical distinction between exact and approximate types, and the monetary calculation trap.

Common Mistakes: Using FLOAT for money. Not knowing that $0.1 + 0.2 \neq 0.3$ in floating-point arithmetic.

Q84: What is the purpose of the RETURNING clause?

Difficulty: Beginner **Category:** DML

Answer: The RETURNING clause (PostgreSQL-specific extension) causes INSERT, UPDATE, or DELETE statements to return values from the affected rows. This eliminates the need for a separate SELECT after a write operation. Examples: INSERT INTO orders (customer_id, amount) VALUES (1, 99.99) RETURNING id, created_at returns the generated id and timestamp. UPDATE employees SET salary = salary * 1.1 WHERE department = 'Eng' RETURNING id, salary returns affected rows and new salaries. DELETE FROM expired_sessions RETURNING user_id returns the ids of deleted sessions. RETURNING * returns all columns. This is especially valuable for getting auto-generated values.

Follow-up Questions:

- Is RETURNING part of the SQL standard?
- How do you use RETURNING with INSERT ON CONFLICT?

What Interviewer Looks For: Practical use cases where RETURNING eliminates a round-trip SELECT, especially for auto-generated IDs.

Common Mistakes: Doing INSERT then a separate SELECT to get the generated ID (inefficient). Not knowing RETURNING is PostgreSQL-specific.

Q85: What is the ISNULL / IFNULL function equivalent in PostgreSQL?

Difficulty: Beginner **Category:** Functions

Answer: PostgreSQL does not have ISNULL or IFNULL (those are from MySQL/SQL Server). The equivalent is COALESCE: COALESCE(value, default_value). Another option is NULLIF which returns NULL if two values are equal. PostgreSQL also has the IS DISTINCT FROM and IS NOT DISTINCT FROM operators that handle NULLs in comparisons: NULL IS NOT DISTINCT FROM NULL is TRUE (unlike NULL = NULL which is NULL/unknown). For conditional NULL replacement, CASE WHEN col IS NULL THEN default ELSE col END is explicit. Best practice is COALESCE for brevity: COALESCE(column, 'default').

Follow-up Questions:

- What is IS DISTINCT FROM?
- How is COALESCE different from CASE WHEN?

What Interviewer Looks For: Knowing PostgreSQL's COALESCE is the equivalent, and awareness of IS DISTINCT FROM for NULL-safe comparisons.

Common Mistakes: Trying to use IFNULL or ISNULL (MySQL syntax) in PostgreSQL. Not knowing IS DISTINCT FROM for NULL-safe equality.

Q86: How do you count rows in a table?

Difficulty: Beginner **Category:** Basic SQL

Answer: `SELECT COUNT()` FROM table_name counts all rows. `SELECT COUNT(column)` FROM table_name counts non-NULL values in that column. For a fast approximate count of large tables, PostgreSQL offers: `SELECT reltuples FROM pg_class WHERE relname = 'table_name'` (updated by `VACUUM/ANALYZE`, may be approximate). `COUNT()` on a large table requires a full table scan and can be slow. Partial counts: `SELECT COUNT(*)` FROM orders WHERE status = 'pending' with a proper index can be fast. For counting distinct values: `SELECT COUNT(DISTINCT column)` FROM table_name. `EXPLAIN` shows the estimated row count without executing the full scan.

Follow-up Questions:

- Why is `COUNT(*)` slow on large tables in PostgreSQL?
- How do you get a fast approximate count?

What Interviewer Looks For: Knowing the difference between `COUNT(*)` and `COUNT(col)`, and awareness of performance alternatives.

Common Mistakes: Using `COUNT(column)` when you want `COUNT(*)` and getting a different result due to NULLs. Not knowing about approximate count via `pg_class`.

Q87: What is a DEFAULT constraint?

Difficulty: Beginner **Category:** Constraints

Answer: A DEFAULT constraint provides an automatic value for a column when no value is specified in the INSERT statement. Syntax: column_name datatype DEFAULT default_value. The default can be a literal value, function call, or expression: `created_at TIMESTAMPTZ DEFAULT NOW(); is_active BOOLEAN DEFAULT TRUE; counter INTEGER DEFAULT 0`. When inserting, you can omit the column and the default is used. Defaults can be added or changed with ALTER TABLE: `ALTER TABLE t ALTER COLUMN col SET DEFAULT 42`. Remove a default: `ALTER TABLE t ALTER COLUMN col DROP DEFAULT`. Defaults are evaluated at INSERT time, so `NOW()` gives the current time for each new row.

Follow-up Questions:

- Can a DEFAULT be a function call?
- How do defaults interact with NULL?

What Interviewer Looks For: Knowing that defaults are evaluated at insert time, and how to add/change defaults with ALTER TABLE.

Common Mistakes: Confusing DEFAULT with NOT NULL (DEFAULT provides a value, NOT NULL ensures it's not null — they work together). Not knowing how to change a default after creation.

Q88: What is the DISTINCT ON clause in PostgreSQL?

Difficulty: Beginner **Category:** Basic SQL

Answer: DISTINCT ON is a PostgreSQL extension that returns one row per distinct value of the specified expression. Unlike regular DISTINCT which deduplicates based on all selected columns, DISTINCT ON lets you pick which column(s) define "distinctness" while returning other columns from that row. Example: `SELECT DISTINCT ON (customer_id) customer_id, order_date, amount FROM orders ORDER BY customer_id, order_date DESC` gets the most recent order per customer. The ORDER BY must begin with the DISTINCT ON columns. It is essentially "get the first row per group" functionality without a subquery. Very useful and more readable than the `ROW_NUMBER()` equivalent.

Follow-up Questions:

- How would you write the DISTINCT ON query using window functions instead?
- What must the ORDER BY clause start with when using DISTINCT ON?

What Interviewer Looks For: Understanding of how DISTINCT ON differs from DISTINCT and its practical use for "latest record per group" queries.

Common Mistakes: Not knowing DISTINCT ON is PostgreSQL-specific. Forgetting that ORDER BY must start with the DISTINCT ON columns.

Q89: What is a lateral join in PostgreSQL?

Difficulty: Beginner **Category:** Joins

Answer: A LATERAL join allows a subquery in the FROM clause to reference columns from preceding table references in the same FROM clause. Without LATERAL, subqueries in FROM cannot reference outer tables. With LATERAL: `SELECT u.name, recent.title FROM users u, LATERAL (SELECT title FROM posts p WHERE p.user_id = u.id ORDER BY created_at DESC LIMIT 3) recent` — this gets the 3 most recent posts per user. LATERAL is similar to a correlated subquery but used in FROM and can return multiple rows. It is very useful for "top N per group" queries and works well with set-returning functions.

Follow-up Questions:

- How is LATERAL different from a correlated subquery?
- When is LATERAL particularly useful?

What Interviewer Looks For: Understanding that LATERAL allows the subquery to reference outer table columns, and "top N per group" use case.

Common Mistakes: Not knowing LATERAL exists. Trying to reference outer tables in a non-LATERAL subquery.

Q90: What is the pg_catalog schema?

Difficulty: Beginner **Category:** Administration

Answer: pg_catalog is a system schema containing all system tables and built-in functions that define PostgreSQL's internal structure. Important pg_catalog tables: pg_class (tables, indexes, sequences), pg_attribute (columns), pg_type (data types), pg_index (index information), pg_constraint (constraints), pg_namespace (schemas), pg_roles (roles/users), pg_database (databases). You can query these to introspect the database: `SELECT * FROM pg_catalog.pg_tables WHERE schemaname = 'public'`. The information_schema provides a SQL-standard alternative. The pg_catalog tables are automatically in the search_path and available in every database.

Follow-up Questions:

- What is the difference between pg_catalog and information_schema?
- How do you list all indexes on a table using pg_catalog?

What Interviewer Looks For: Awareness of system catalogs for database introspection and key catalog tables.

Common Mistakes: Not knowing about system catalogs for metadata queries. Using application-level knowledge instead of querying pg_catalog.

Q91: What is the information_schema?

Difficulty: Beginner **Category:** Administration

Answer: information_schema is a SQL standard schema containing views that provide metadata about the database in a standardized way. Key views: information_schema.tables (list of tables), information_schema.columns (column definitions), information_schema.table_constraints (constraints), information_schema.key_column_usage (FK/PK columns), information_schema.referential_constraints (foreign keys). Unlike pg_catalog, information_schema is portable across different database systems. It is slower than pg_catalog for some queries but provides better compatibility. Example: SELECT column_name, data_type FROM information_schema.columns WHERE table_name = 'employees'.

Follow-up Questions:

- Which should you use: pg_catalog or information_schema?
- How do you find all foreign keys on a table using information_schema?

What Interviewer Looks For: Understanding the portability vs performance tradeoff between information_schema and pg_catalog.

Common Mistakes: Not knowing information_schema exists. Not knowing the difference between the two metadata systems.

Q92: How do you use EXPLAIN ANALYZE to identify slow queries?

Difficulty: Beginner **Category:** Performance

Answer: EXPLAIN ANALYZE executes the query and shows actual execution statistics. Look for: 1) Seq Scan on large tables where an Index Scan would be faster; 2) high "actual rows" much larger than "rows=" estimate (bad statistics — run ANALYZE); 3) nested loop joins on large tables (should be hash or merge joins); 4) sort nodes on large datasets without an index; 5) nodes with high "actual time" relative to others. The output is a tree: read bottom-up, as inner nodes execute first. BUFFERS option (EXPLAIN (ANALYZE, BUFFERS)) shows cache hits vs disk reads. Wrap in BEGIN/ROLLBACK when using with DML to avoid side effects.

Follow-up Questions:

- What does the "cost" in an execution plan represent?
- How do you read a nested loop join in an execution plan?

What Interviewer Looks For: Practical ability to read a plan and identify performance issues.

Common Mistakes: Running EXPLAIN ANALYZE on production without wrapping DML in a transaction. Not using the BUFFERS option for I/O analysis.

Q93: What is the difference between TRUNCATE and DELETE in terms of MVCC?

Difficulty: Beginner **Category:** Advanced

Answer: DELETE removes rows one at a time using MVCC — each deleted row is marked as dead but kept for any concurrent transactions that might still see it. Old versions are cleaned up by VACUUM later. TRUNCATE removes all rows by replacing the entire relation file, essentially creating new empty heap files. This bypasses per-row MVCC machinery. Because TRUNCATE doesn't create individual dead tuples, VACUUM doesn't need to

clean up after it. However, TRUNCATE must acquire an ACCESS EXCLUSIVE lock and is more "visible" to concurrent transactions. Despite bypassing MVCC per-row tracking, TRUNCATE is still transactional in PostgreSQL.

Follow-up Questions:

- Can a concurrent SELECT see data after TRUNCATE is rolled back?
- Why does TRUNCATE require an exclusive lock?

What Interviewer Looks For: Understanding of the file-level vs row-level operation and its MVCC implications.

Common Mistakes: Thinking TRUNCATE and DELETE both leave dead tuples. Not knowing TRUNCATE is transactional in PostgreSQL.

Q94: What is the COPY command's performance advantage?

Difficulty: Beginner **Category:** Performance

Answer: COPY is significantly faster than individual INSERT statements for bulk data loading. The performance difference comes from: 1) COPY bypasses most per-row overhead and uses optimized internal data paths; 2) With COPY, the whole batch is processed in a single transaction, whereas many INSERTs require transaction overhead per statement; 3) COPY can disable index maintenance during load (not directly, but combined with UNLOGGED tables and post-load index creation); 4) COPY writes WAL more efficiently. In practice, COPY can be 5-100x faster than equivalent INSERT statements for large datasets. For best performance, combine with: disabling triggers, dropping indexes before load, using UNLOGGED tables, then rebuild.

Follow-up Questions:

- How would you load a billion rows into PostgreSQL efficiently?
- What is an UNLOGGED table?

What Interviewer Looks For: Understanding the mechanics of why COPY is faster and best practices for bulk loading.

Common Mistakes: Using repeated INSERTs for large data loads. Not knowing about UNLOGGED tables for staging data.

Q95: What are the different types of indexes in PostgreSQL?

Difficulty: Beginner **Category:** Indexes

Answer: PostgreSQL supports several index types: B-tree (default) — balanced tree, works for =, <, >, range queries, BETWEEN, LIKE 'prefix%', and sorting; Hash — only for equality (=), smaller than B-tree for simple equality; GiST (Generalized Search Tree) — used for geometric types, full-text search, nearest-neighbor; SP-GiST (Space-Partitioned GiST) — for non-balanced structures like quadrees; GIN (Generalized Inverted Index) — used for full-text search, JSONB, arrays; BRIN (Block Range INdex) — very small index for naturally ordered data (timestamps, sequences). B-tree is appropriate for most cases. GIN is ideal for JSONB and full-text search.

Follow-up Questions:

- When would you use a GIN index vs a B-tree?
- What is a BRIN index best suited for?

What Interviewer Looks For: Knowing the main index types and their appropriate use cases.

Common Mistakes: Using B-tree for JSONB containment queries when GIN is needed. Not knowing BRIN exists for large, naturally ordered tables.

Q96: What is a partial index?

Difficulty: Beginner **Category:** Indexes

Answer: A partial index is an index built on a subset of rows, defined by a WHERE clause. Example: CREATE INDEX idx_orders_pending ON orders (created_at) WHERE status = 'pending'. This index only includes rows where status = 'pending', making it smaller and faster to maintain than a full index. Partial indexes are excellent when: you only query a small subset of rows frequently (e.g., only active records); you want to enforce uniqueness on a subset (e.g., unique email for active users only). They save disk space and write overhead. A query can only use a partial index if its WHERE clause is compatible with the index predicate.

Follow-up Questions:

- How does the query planner decide to use a partial index?
- How would you create a unique partial index?

What Interviewer Looks For: Understanding of the performance benefit and the compatibility requirement between query and index predicates.

Common Mistakes: Not knowing partial indexes exist. Creating a full index when only a small subset is ever queried.

Q97: What is connection string format in PostgreSQL?

Difficulty: Beginner **Category:** Administration

Answer: PostgreSQL connection strings can be in keyword=value format or URI format. Keyword format: host=localhost port=5432 dbname=mydb user=myuser password=mypassword. URI format: postgresql://myuser:mypassword@localhost:5432/mydb or postgres://myuser:mypassword@localhost:5432/mydb. Additional options can be appended: postgresql://user@host/db?sslmode=require&connect_timeout=10. Environment variables can be used instead: PGHOST, PGPORT, PGDATABASE, PGUSER, PGPASSWORD. The .pgpass file stores passwords securely. SSL mode options: disable, allow, prefer, require, verify-ca, verify-full.

Follow-up Questions:

- How do you configure SSL for a PostgreSQL connection?
- What is the .pgpass file?

What Interviewer Looks For: Knowing both connection string formats and environment variable alternatives.

Common Mistakes: Hardcoding passwords in connection strings. Not knowing about PGPASSWORD environment variable or .pgpass file.

Q98: What is ANALYZE in PostgreSQL?

Difficulty: Beginner **Category:** Maintenance

Answer: ANALYZE collects statistics about the contents of tables, storing them in the pg_statistic catalog. The query planner uses these statistics to choose efficient query plans — estimates of row counts, value distributions, and correlations. Without up-to-date statistics, the planner may make poor choices (e.g.,

choosing a sequential scan when an index would be faster). ANALYZE runs automatically as part of autovacuum. Run manually: ANALYZE table_name or just ANALYZE (all tables). VACUUM ANALYZE does both in one pass. After bulk data loads, running ANALYZE manually ensures the planner has accurate statistics immediately rather than waiting for autovacuum.

Follow-up Questions:

- What happens to query performance if statistics are stale?
- What does autovacuum's role with statistics?

What Interviewer Looks For: Understanding of why statistics matter for query planning and when to run ANALYZE manually.

Common Mistakes: Not running ANALYZE after large data loads. Not knowing that stale statistics cause bad query plans.

Q99: What is a composite primary key?

Difficulty: Beginner **Category:** Constraints

Answer: A composite primary key is a primary key consisting of two or more columns where the combination of those columns uniquely identifies each row. Example: a junction table order_items might have a composite PK of (order_id, product_id). Neither column alone is unique, but the combination is. Syntax: CREATE TABLE order_items (order_id INTEGER REFERENCES orders(id), product_id INTEGER REFERENCES products(id), quantity INTEGER, PRIMARY KEY (order_id, product_id)). Composite PKs are common in many-to-many relationship tables. They can be referenced by composite foreign keys.

Follow-up Questions:

- When would you use a composite primary key vs a surrogate key?
- How do you define a composite foreign key?

What Interviewer Looks For: Understanding of the syntax and common use cases (junction tables), plus the surrogate key tradeoff.

Common Mistakes: Always adding a surrogate key when a natural composite key is sufficient. Not knowing how to write a composite FK.

Q100: How do you write an INSERT ON CONFLICT (upsert) statement?

Difficulty: Beginner **Category:** DML

Answer: INSERT ON CONFLICT is PostgreSQL's upsert mechanism, introduced in PostgreSQL 9.5. It handles the case where an INSERT would violate a unique constraint: INSERT INTO users (id, email, name) VALUES (1, 'a@b.com', 'Alice') ON CONFLICT (id) DO UPDATE SET email = EXCLUDED.email, name = EXCLUDED.name. The EXCLUDED table refers to the row that was proposed for insertion. ON CONFLICT DO NOTHING ignores the conflict and skips insertion. This is atomic and avoids race conditions that would occur with a separate SELECT then INSERT approach. The conflict target (ON CONFLICT (column) or ON CONFLICT ON CONSTRAINT) specifies which constraint violation to handle.

Follow-up Questions:

- What is the EXCLUDED table in ON CONFLICT?
- How is upsert different from MERGE?

What Interviewer Looks For: Understanding of EXCLUDED, the atomic nature, and knowing ON CONFLICT DO NOTHING.

Common Mistakes: Not knowing EXCLUDED represents the incoming row values. Using a SELECT-then-INSERT pattern that has race conditions.

End of 01_beginner_questions.md — 100 Questions